

Introdução a Algoritmos

Aula 5 – Repetição

Raoni F. S. Teixeira · 2s/2026

Objetivos desta aula

Ao final desta aula, você deverá ser capaz de:

1. Aplicar as três perguntas de um laço (o que se repete, quantas vezes, o que muda) para planejar a estrutura de while, do-while e for;
2. Implementar os padrões de acumulação e contagem dentro de laços;
3. Identificar e corrigir um loop infinito causado pela ausência do avanço na variável de controle;
4. Explicar a equivalência entre for e while e escolher a forma mais legível para cada situação.

1 A peça “repetição”

Nas aulas anteriores, montamos programas que executam instruções em sequência e tomam decisões. Com essas duas peças, já conseguimos descrever uma grande variedade de algoritmos. Mas ainda falta algo essencial: a capacidade de **repetir** um bloco de instruções sem precisar escrevê-lo várias vezes.

Considere o problema de imprimir os números de 1 a 100. Com o que sabemos até agora, a única solução seria escrever cem `printfs` — o que é impraticável. Pior: e se quisermos imprimir de 1 a **n**, onde **n** é lido do teclado? Nesse caso, é impossível escrever um programa finito que funcione para qualquer **n** sem usar repetição.

A peça “repetição” — chamada de **laço** ou **loop** — resolve esse problema. Ela permite que um bloco de instruções seja executado enquanto uma condição for verdadeira, podendo repetir zero, uma ou milhões de vezes dependendo dos dados.

C oferece três formas de laço: `while`, `do-while` e `for`. As três são equivalentes em poder de expressão — qualquer problema que se resolve com uma delas pode ser resolvido com as outras duas. A diferença é apenas de forma: o `for` agrupa os três elementos do laço numa linha só, o que é conveniente em muitas situações; o `do-while` garante que o bloco execute ao menos uma vez, invertendo a posição do teste. Mas não existe uma regra que diga “use `for` para isso e `while` para aquilo” — use o que deixar o código mais claro.

2 As três perguntas de um laço

Antes de escrever qualquer laço, responda três perguntas:

Pergunta. **O que deve ser repetido?** — Qual é o bloco de instruções que se repete?

Perguntb. **Quantas vezes (ou até quando) deve se repetir?** — Qual é a condição de parada?

Perguntc. **O que muda a cada repetição?** — Como o estado do programa evolui para que a condição de parada eventualmente seja alcançada?

A terceira pergunta é crucial. Um laço cujo estado nunca muda **nunca termina** — o programa trava num **loop infinito**, um dos bugs mais frustrantes de depurar.

3 Comando while

O while é o laço mais fundamental: testa a condição **antes** de cada execução do bloco. Se a condição for falsa na primeira vez, o bloco nunca é executado.

```
while (condicao) {  
    /* bloco repetido enquanto condicao for verdadeira */  
}
```

O fluxo é:

1. Avalia a condição;
2. Se verdadeira, executa o bloco e volta ao passo 1;
3. Se falsa, encerra o laço.

Exemplo — Números de 1 a 100

Aplicando as três perguntas:

- O que se repete? → imprimir o número atual.
- Até quando? → enquanto o número for ≤ 100 .
- O que muda? → o número incrementa a cada volta.

```
int i = 1;  
while (i <= 100) {  
    printf("%d ", i);  
    i++;  
}
```

A variável *i* cumpre três papéis: **inicia** antes do laço ($i = 1$), **controla** a condição ($i \leq 100$) e **avança** dentro do bloco ($i++$). Todo laço bem formado tem esses três elementos.

Exemplo — Soma dos números de 1 a *n*

```
int i = 1, n, soma = 0;  
scanf("%d", &n);  
while (i <= n) {  
    soma += i;  
    i++;  
}  
printf("Soma: %d\n", soma);
```

Observe o padrão de **acumulação**: a variável *soma* começa em zero e vai recebendo valores ao longo do laço. Esse padrão aparece em incontáveis algoritmos — reconhecê-lo acelera muito o raciocínio.

3.1 Loop infinito: o laço que não termina

```
int i = 1;
while (i <= 10) {
    printf("%d\n", i);
    /* esquecemos o i++ - i nunca muda, laço infinito! */
}
```

Se o estado da variável de controle nunca for alterado dentro do laço, a condição sempre permanece verdadeira e o programa nunca termina. No terminal, você pode interrompê-lo com Ctrl+C.

Atenção

Todo laço precisa de **três elementos obrigatórios**:

1. **Inicialização** — a variável de controle recebe um valor inicial antes do laço.
2. **Condição de parada** — a expressão que determina se o laço continua.
3. **Avanço** — pelo menos uma instrução dentro do bloco que modifica o estado e aproxima o programa do término.

A ausência de qualquer um desses elementos quase sempre resulta em loop infinito ou em comportamento incorreto.

3.2 Padrões comuns com while

3.2.1 Contar até um valor lido

```
int i = 1, n;
scanf("%d", &n);
while (i <= n) {
    printf("%d\n", i);
    i++;
}
```

3.2.2 Repetir até o usuário decidir parar

```
int continua = 1, valor;
while (continua) {
    printf("Digite um valor (0 para sair): ");
    scanf("%d", &valor);
    if (valor == 0)
        continua = 0;
    else
        printf("Você digitou: %d\n", valor);
}
```

3.2.3 Divisão inteira por subtração sucessiva

Calcular a / b usando apenas subtração — uma forma de entender o que divisão realmente significa:

```
int dividendo, divisor, quociente = 0, resto;
scanf("%d %d", &dividendo, &divisor);
resto = dividendo;
while (resto >= divisor) {
    resto -= divisor;
    quociente++;
}
```

```
    quociente++;  
}  
printf("%d / %d = %d (resto %d)\n", dividendo, divisor, quociente, resto);
```

4 Comando do-while

O do-while é uma variação do while com uma diferença fundamental: a condição é testada **depois** de cada execução do bloco. Isso garante que o bloco seja executado **pelo menos uma vez**, independentemente da condição.

```
do {  
    /* bloco executado ao menos uma vez */  
} while (condicao);
```

4.1 Quando usar do-while?

O do-while é a escolha natural quando precisamos executar a ação antes de saber se ela deve se repetir — o caso clássico é **validação de entrada**: primeiro lemos o valor, depois verificamos se é válido.

Exemplo — Validar entrada do usuário

Com while, precisamos de código duplicado ou de um valor inicial artificial:

```
/* Com while – awkward: lemos n antes do laço só para inicializar */  
int n;  
scanf("%d", &n);  
while (n < 1 || n > 10) {  
    printf("Valor inválido. Digite entre 1 e 10: ");  
    scanf("%d", &n);  
}
```

Com do-while, o fluxo é mais natural:

```
/* Com do-while – lemos, depois verificamos */  
int n;  
do {  
    printf("Digite um valor entre 1 e 10: ");  
    scanf("%d", &n);  
} while (n < 1 || n > 10);  
printf("Valor aceito: %d\n", n);
```

5 Comando for

O for é o laço mais usado quando o número de repetições é conhecido de antemão. Ele compacta os três elementos obrigatórios do laço — inicialização, condição e avanço — numa única linha.

```
for (inicializacao; condicao; avanco) {  
    /* bloco repetido */  
}
```

O fluxo é idêntico ao do while:

1. Executa a **inicialização** (apenas uma vez, antes do laço);
2. Avalia a **condição** — se falsa, encerra;
3. Executa o **bloco**;

4. Executa o **avanço** e volta ao passo 2.

De fato, o for e o while são equivalentes:

```
/* for */
for (int i = 1; i <= n; i++) {
    printf("%d\n", i);
}
```

```
/* while equivalente */
int i = 1;
while (i <= n) {
    printf("%d\n", i);
    i++;
}
```

O for é preferido quando os três elementos são simples e se encaixam na mesma linha. O while é preferido quando a inicialização é complexa ou quando a condição de parada não envolve um contador.

Exemplo — Tabela de potências de 2

```
int i, pot;
int n;
printf("Quantas potências? ");
scanf("%d", &n);

for (i = 1, pot = 2; i <= n; i++, pot *= 2) {
    printf("2^%d = %d\n", i, pot);
}
```

Exemplo — Laço decrescente

O for não precisa ser crescente — podemos percorrer qualquer sequência:

```
for (int i = 10; i >= 1; i--) {
    printf("%d\n", i);    /* conta de 10 a 1 */
}
```

Exemplo — Laço com passo diferente de 1

```
for (int i = 0; i <= 100; i += 5) {
    printf("%d\n", i);    /* 0, 5, 10, 15, ..., 100 */
}
```

6 Laços aninhados

É possível colocar um laço dentro do bloco de outro laço. Para cada iteração do laço externo, o laço interno executa **completamente**.

Exemplo — Tabuada de multiplicação

```
for (int i = 1; i <= 9; i++) {
    for (int j = 1; j <= 9; j++) {
        printf("%3d", i * j);
    }
}
```

```
    }  
    printf("\n");  
}
```

O `%3d` formata o número num campo de 3 caracteres, alinhando as colunas. Para `i=1`: `j` percorre 1..9, imprimindo os produtos. Depois, uma quebra de linha. Para `i=2`: idem. E assim por diante até `i=9`.

Atenção

Laços aninhados multiplicam o número de iterações. Dois laços de n iterações cada resultam em n^2 execuções do bloco mais interno. Para $n = 1000$, isso é um milhão de operações — o que pode ser lento. Tenha consciência da complexidade dos laços aninhados que você escreve.

7 for, while e do-while: qual a diferença real?

`for` e `while` são completamente intercambiáveis — um pode sempre ser reescrito como o outro sem nenhuma perda. A escolha é puramente uma questão de clareza: o `for` é conveniente quando inicialização, condição e avanço cabem juntos numa linha e formam uma unidade coesa; o `while` é conveniente quando esses elementos estão espalhados ou quando não há um contador explícito.

O único laço com semântica distinta é o `do-while`: ele **garante** que o bloco execute ao menos uma vez, pois o teste vem depois. Isso não pode ser reproduzido diretamente com `while` sem duplicar código ou usar uma variável auxiliar — como mostrado no exemplo da validação de entrada.

Não memorize regras do tipo “use `for` para contadores e `while` para condições gerais”. Escreva o laço que deixar sua intenção mais legível. Com prática, a escolha se tornará instintiva.

8 Exemplo integrador

O programa abaixo combina todas as estruturas estudadas nesta aula: lê números do usuário até que ele decida parar, depois calcula e exibe a média.

```
#include <stdio.h>  
  
/*  
 * Descrição: Lê uma sequência de notas até o usuário digitar -1  
 *           e exibe a média das notas válidas.  
 * Entrada:  sequência de reais; -1 como sentinela  
 * Saída:    média das notas, ou mensagem se nenhuma nota for fornecida  
 */  
int main() {  
    float nota, soma = 0.0;  
    int quantidade = 0;  
  
    printf("Digite as notas (-1 para encerrar):\n");  
  
    /* do-while: lemos ao menos uma nota antes de verificar */  
    do {  
        scanf("%f", &nota);  
        if (nota != -1) {
```

```
        soma += nota;
        quantidade++;
    }
} while (nota != -1);

if (quantidade == 0) {
    printf("Nenhuma nota fornecida.\n");
} else {
    printf("Média: %.2f\n", soma / quantidade);
}

return 0;
}
```

9 Exercícios

Para cada exercício, responda explicitamente as três perguntas (o que se repete, até quando, o que muda) antes de escrever o código.

1. Escreva um programa que imprima todos os números pares de 1 até n (n lido do teclado). Implemente duas vezes: uma com `while` e outra com `for`. Verifique que o comportamento é idêntico.
2. Escreva um programa que calcule o fatorial de n (n lido do teclado). (*Lembre: $0! = 1$ por definição — trate esse caso.*)
3. Escreva um programa que leia repetidamente um número inteiro e some-o a um acumulador, parando quando o usuário digitar 0. Ao final, exiba a soma. Implemente com `while` e depois reescreva com `do-while`. Os dois programas se comportam da mesma forma? Há alguma diferença?
4. Escreva um programa que imprima o triângulo abaixo para $n = 5$ (n lido do teclado). Use laços aninhados.

```
*
**
***
****
*****
```

5. (*Desafio*) Escreva um programa que verifique se um número inteiro positivo é primo. Um número é primo se for divisível apenas por 1 e por ele mesmo. (*Dica: teste divisores de 2 até $\sqrt{n}$.*)